

Documentation technique — DataShare

Point d'entrée concis de la documentation technique. Chaque section renvoie au document détaillé correspondant.

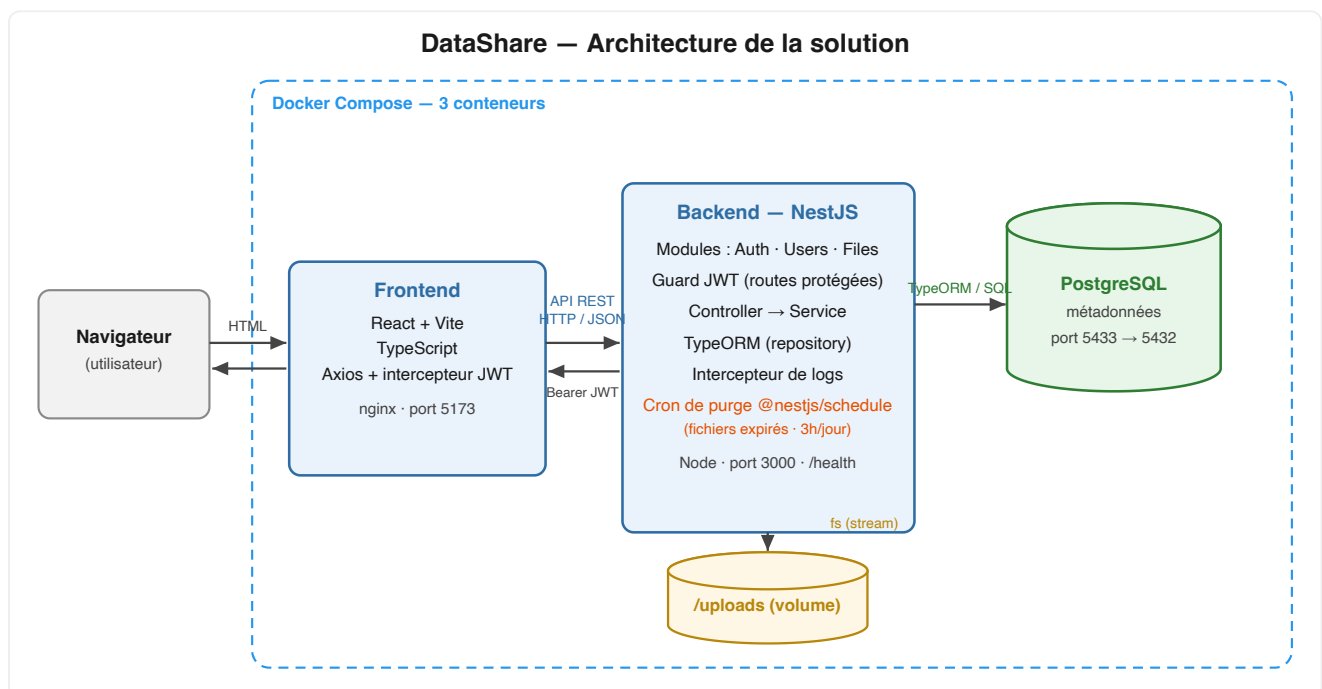
DataShare est une application web de **partage de fichiers par lien temporaire** : un utilisateur dépose un fichier, l'application génère un lien unique (protégeable par mot de passe) qui expire au bout de 1 à 7 jours.

1. Architecture de l'application

Architecture **3 couches**, entièrement conteneurisée :

- **Frontend** React + Vite (nginx, port 5173) → **Backend** NestJS (port 3000) → **PostgreSQL** (port 5433).
- Le front parle au back via une **API REST** (HTTP/JSON), authentifié par un **cookie JWT HttpOnly** ; le back lit/écrit les métadonnées en base via TypeORM et stocke les fichiers sur disque (`/uploads`) en streaming. Une tâche planifiée (cron) purge les fichiers expirés.

Diagramme et trajet détaillé d'une requête : **ARCHITECTURE.md** (image + source Mermaid).



2. Justification des choix technologiques

Tous les choix de stack sont pris **parmi les options imposées** ; voici le pourquoi de chacun et les alternatives écartées.

Backend — NestJS (TypeScript)

- **Pourquoi** : impose une **architecture modulaire** (modules / contrôleurs / services), intègre nativement TypeScript, l'**injection de dépendances**, les tâches planifiées (`@nestjs/schedule`) et les tests.
- **Validation des entrées** : DTO + **class-validator** validés automatiquement par un `ValidationPipe` global (rejet en 400 avant le contrôleur).
- **Documentation d'API** : **Swagger UI** générée automatiquement, servie sur `/api`.
- **Versionnage** : routes métier sous `/v1` (préparer les évolutions sans casser l'existant).
- **Alternatives** : Spring (Java, plus lourd à mettre en place pour un MVP), Symfony (PHP), Express (trop minimal, peu de structure imposée).
- **Compromis** : courbe d'apprentissage des décorateurs, compensée par un code organisé et testable.

Langage — TypeScript (front et back)

- **Pourquoi** : le **typage** détecte les erreurs avant l'exécution (mauvais type, faute de frappe), précieux sur un projet API + base de données. Code plus clair et maintenable.
- **Alternative** : JavaScript pur — écarté pour la sûreté de typage.

Frontend — React + Vite

- **Pourquoi** : React est le **standard du marché** pour les interfaces dynamiques, large écosystème. **Vite** offre un démarrage et un build très rapides (HMR instantané, bundle optimisé par tree-shaking).
- **Accessibilité & responsive** : styles en **CSS Modules** avec *media queries* (mobile/desktop), focus clavier visible, `label` / `input` associés, éléments interactifs sémantiques (`button` , `aria-label`) — conformité WCAG/ARIA de base.
- **État applicatif** : géré via **React Context** (connexion/déconnexion), sans *prop drilling*.
- **Alternatives** : Angular (plus structurant mais plus verbeux), Vue (écosystème plus restreint).

Base de données — PostgreSQL

- **Pourquoi** : base **relationnelle** robuste et gratuite, adaptée à des données liées (« un utilisateur possède plusieurs fichiers »). Garantit les **relations** (clé étrangère) et l'**unicité** (email unique).
- **Alternative** : MongoDB (NoSQL) — écarté car le modèle est clairement relationnel.

ORM — TypeORM

- **Pourquoi** : intégration native NestJS ; mes classes annotées `@Entity` deviennent des tables, je manipule des objets plutôt que du SQL à la main.
- **Compromis** : `synchronize: true` est pratique en dev mais à remplacer par des **migrations** en production.

Authentification — JWT (cookie HttpOnly) + bcrypt

- **Pourquoi** : **JWT** est *stateless* (pas de session serveur à stocker, scalable) ; le serveur signe un jeton et vérifie sa signature. **bcrypt** hache les mots de passe (à sens unique, lent par conception → résistant à la force brute).
- **Transport sécurisé** : le JWT est déposé dans un **cookie HttpOnly** (illisible par JavaScript → protège du vol par XSS), avec repli sur l'en-tête `Authorization: Bearer` pour les clients API.
- **Alternative** : sessions serveur (état à maintenir), OAuth (surdimensionné pour le MVP).

Stockage des fichiers — disque + métadonnées en base

- **Pourquoi** : les **fichiers** vont sur le disque (`/uploads`), seules les **métadonnées** (nom, taille, token, expiration) en base. Stocker les fichiers en base serait lourd et lent.
- **Réception** : **Multer** (middleware d'upload), avec une limite de taille de 1 Go.

Conteneurisation — Docker Compose

- **Pourquoi** : un **environnement identique partout** et un déploiement reproductible (fin du « ça marche sur ma machine »). 3 services orchestrés : PostgreSQL, backend, frontend.

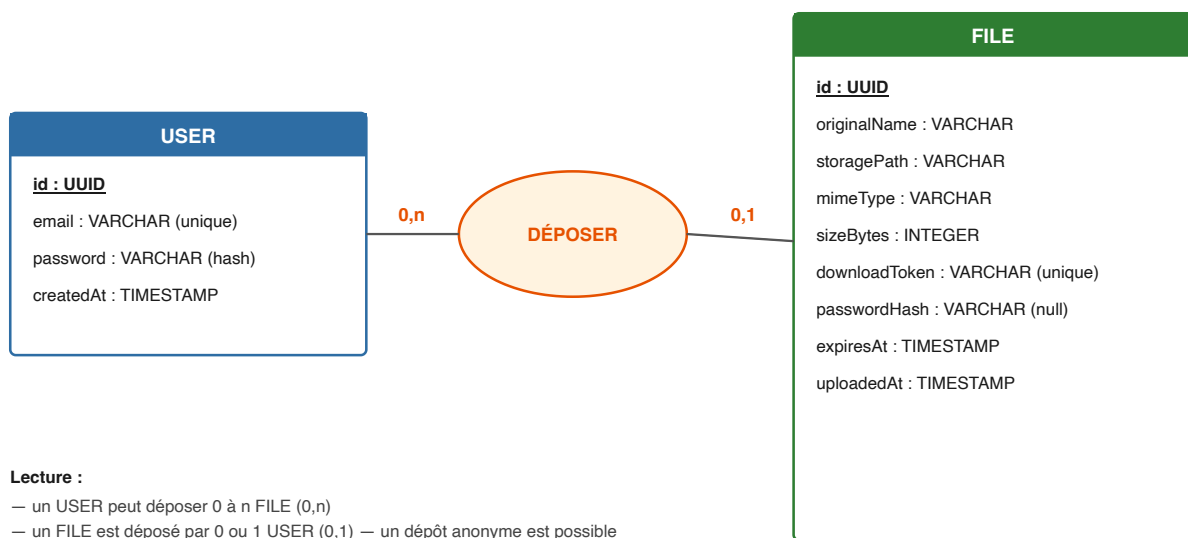
Tests — Jest, Supertest, Cypress, k6

- **Pourquoi** : **Jest** (unitaire, livré avec NestJS), **Supertest** (intégration HTTP), **Cypress** (end-to-end navigateur), **k6** (test de charge). Couvre la pyramide des tests + la performance.

3. Modèle de données

Deux entités : **USER** (1) — (n) **FILE**, reliées par une clé étrangère `user_id` (nullable, `ON DELETE CASCADE`). MCD complet (formalisme Merise) + tables détaillées : **MODELE-DE-DONNEES.md**.

DataShare — Modèle Conceptuel de Données (MCD)



Traduction physique : la table files porte une clé étrangère user_id (NULL autorisé, ON DELETE CASCADE).

4. Documentation des endpoints (API)

Contrat d'interface complet au format **OpenAPI 3.0.3** : [../openapi.yaml](#). Documentation **interactive Swagger** générée et servie sur <http://localhost:3000/api>.

Les routes métier sont **versionnées sous** `/v1` ; `/health` et les routes de téléchargement public `/d/...` restent **non versionnées** (liens partageables stables).

Méthode	Route	Rôle	Auth
GET	<code>/health</code>	Health check (non versionné)	—
POST	<code>/v1/auth/register</code>	Créer un compte (pose le cookie)	—
POST	<code>/v1/auth/login</code>	Se connecter (pose le cookie JWT)	—
POST	<code>/v1/auth/logout</code>	Se déconnecter (efface le cookie)	cookie
POST	<code>/v1/files/upload</code>	Téléverser un fichier	cookie/JWT
GET	<code>/v1/files</code>	Lister mes fichiers (filtre + pagination <code>page</code> / <code>limit</code>)	cookie/JWT
DELETE	<code>/v1/files/:id</code>	Supprimer un de mes fichiers	cookie/JWT
GET	<code>/d/:token</code>	Infos d'un lien de partage (non versionné)	—
POST	<code>/d/:token/download</code>	Télécharger le fichier (non versionné)	—

5. Sécurité et gestion des accès

Mots de passe hachés (bcrypt), **JWT dans un cookie HttpOnly** (anti-XSS) signé avec un secret en variable d'environnement, validation des entrées (class-validator), **CORS restreint via variable d'environnement** (`CORS_ORIGIN`), liens = UUID non devinables + expiration, refus des exécutables (magic number), cloisonnement par propriétaire, limite de taille (anti-DoS). **Conformité RGPD** (minimisation, durées de conservation, droit à l'effacement) et **suivi des vulnérabilités** `npm audit` documentés dans `../SECURITY.md`.

6. Qualité, tests et maintenance

58 tests (47 unitaires Jest + 7 intégration Supertest + 4 e2e Cypress), couverture **98,7 %** (logique métier à 100 %), test de charge k6 (login + upload/download) et budget de performance front (Lighthouse). **Intégration continue GitHub Actions** : les tests et le build sont rejoués à chaque push. Logs structurés JSON. Détail : `../TESTING.md` · performance `../PERF.md` · maintenance `../MAINTENANCE.md`.

7. Installation et exécution

Prérequis : Node 20+, Docker. Démarrage rapide :

```
./setup.sh                # base PostgreSQL + dépendances
cd backend && npm run start:dev # backend (:3000)
cd frontend && npm run dev    # frontend (:5173)
```

Ou tout en conteneurs : `docker-compose up -d --build` . Détail complet : `../README.md`.

8. Utilisation de l'IA dans le développement

Tâches confiées, rôle de supervision et correctifs apportés : `../AI_USAGE.md`.